

# Arquitectura del Sistema — UAO Finder

---

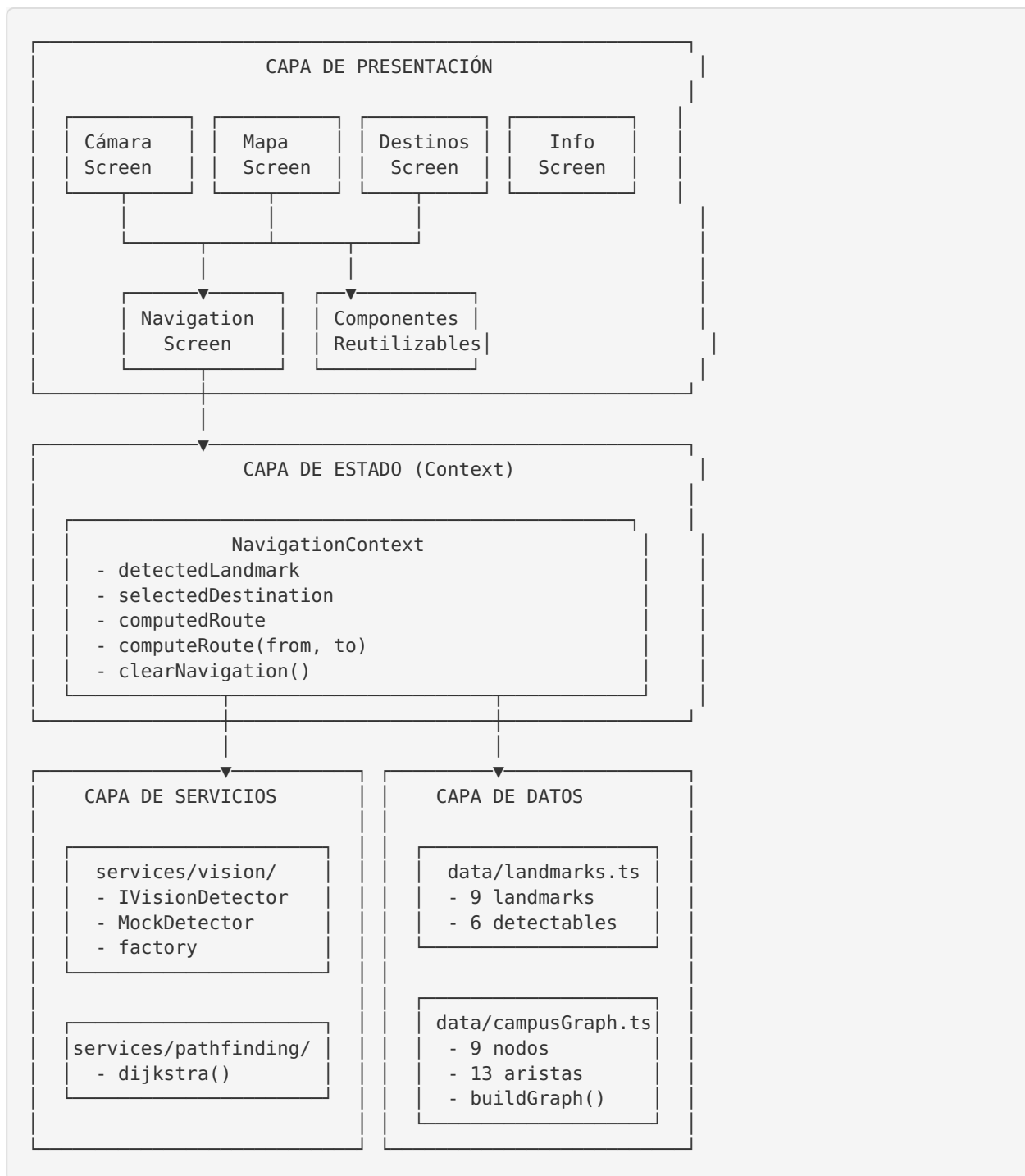
## Descripción General

---

UAO Finder sigue una arquitectura modular por capas diseñada para facilitar la mantenibilidad, testabilidad y la futura integración de un modelo real de visión por computadora.

---

## Diagrama de Flujo de Datos



## Descripción de Módulos

### /src/services/vision/

Módulo de visión por computadora con patrón Strategy:

- **types.ts** : Define la interfaz `IVisionDetector` con el método `detect(imageData?)` . Cualquier implementación (mock o real) debe cumplir este contrato.

- **MockDetector.ts** : Implementación simulada que retorna landmarks aleatorios con confianza entre 20-99%. Simula un delay de 800ms para emular el tiempo de inferencia.
- **factory.ts** : Función factory `createVisionDetector()` que retorna la implementación activa. Para integrar un modelo real de TFLite, solo se necesita crear una clase `RealDetector` que implemente `IVisionDetector` y cambiar la factory.

### /src/services/pathfinding/

- **dijkstra.ts** : Implementación pura en TypeScript del algoritmo de Dijkstra. Recibe el grafo del campus, nodo origen y nodo destino. Retorna la ruta más corta con:
  - Lista ordenada de nodos del camino
  - Distancia total en metros
  - Tiempo estimado de caminata (a 80m/min)
  - Instrucciones paso a paso en español

### /src/data/

- **landmarks.ts** : Definiciones de los 9 puntos del campus con coordenadas mock (x, y), categoría, descripción en español e icono.
- **campusGraph.ts** : Define las 13 aristas del grafo con distancias en metros y descripciones de dirección en español (tanto en sentido directo como inverso). La función `buildCampusGraph()` construye el grafo como mapa de adyacencia.

### /src/context/

- **NavigationContext.tsx** : React Context que centraliza el estado de navegación:
  - Landmark detectado por la cámara
  - Destino seleccionado por el usuario
  - Ruta computada (resultado de Dijkstra)
  - Métodos para calcular rutas y limpiar el estado

### /src/components/

Componentes reutilizables:

- **CampusMapView** : Mapa SVG interactivo con nodos, aristas y visualización de rutas
  - **DetectionOverlay** : Overlay para mostrar resultados de detección en la cámara
  - **LandmarkCard** : Tarjeta de destino con icono, nombre, categoría y descripción
  - **RouteSteps** : Componente stepper vertical para instrucciones paso a paso
  - **SearchBar** : Barra de búsqueda con filtrado en tiempo real
  - **ErrorBoundary** : Captura errores de renderizado con mensaje en español
-

## Stack Tecnológico

| Componente | Tecnología                                 |
|------------|--------------------------------------------|
| Framework  | React Native 0.81                          |
| Plataforma | Expo SDK 54                                |
| Lenguaje   | TypeScript 5.x                             |
| Navegación | expo-router (file-based routing)           |
| Mapa       | react-native-svg                           |
| Cámara     | expo-camera                                |
| UI         | react-native-paper, expo-linear-gradient   |
| Fuentes    | Montserrat (Google Fonts), Open Sans       |
| Estado     | React Context + Hooks                      |
| Algoritmos | Dijkstra (implementación propia)           |
| Detección  | Faster R-CNN (mock, preparado para TFLite) |

## Consideraciones de Diseño

1. **Modularidad:** El patrón Strategy en el módulo de visión permite intercambiar detectores sin modificar el resto del sistema.
2. **Offline-first:** Todo el procesamiento (grafo, Dijkstra, mock detection) se ejecuta localmente sin necesidad de conexión a internet.
3. **Internacionalización:** Toda la interfaz y las instrucciones de navegación están en español.
4. **Coordenadas reemplazables:** Las coordenadas (x, y) de los landmarks son mock y están documentadas como reemplazables por coordenadas GPS reales.
5. **Accesibilidad:** Todos los elementos interactivos tienen `accessibilityLabel` en español, cumpliendo con un ratio de contraste  $\geq 4.5:1$ .